

DECODE_2_2

Reconstruction sous-clonale par mélange de processus de Dirichlet de Beta-Binomiales,
inférence variationnelle en champ moyen

Note de méthodes

June 26, 2026

Abstract

DECODE_2_2 reconstruit l'architecture sous-clonale d'une tumeur à partir d'un *unique* échantillon de séquençage ADN bulk : il infère *combien* de sous-clones existent, *quelle fraction de cellules* chacun occupe, et *quelle mutation appartient à quel clone*, directement à partir des comptages de reads. Le moteur est un *mélange de processus de Dirichlet tronqué d'émissions Beta-Binomiales*, dont la fraction cellulaire (CCF) de chaque composante est représentée par une catégorielle sur une grille fixe (l'astuce de tractabilité de PyClone-VI), ajusté par *inférence variationnelle en champ moyen* (montée par coordonnées, CAVI). Une composante de *queue neutre* en loi de puissance ($1/f$, style MOBSTER) absorbe le continuum mutationnel neutre pour éviter le sur-découpage ; la pureté est *inférée* en maximisant l'ELBO ; et le nombre de clusters est choisi par *fusion agglomérative guidée par l'ICL*, « N-aware » et à entropie tempérée. L'ensemble est déterministe, ne dépend que de `numpy/scipy`, et tourne en quelques minutes pour les 51 tumeurs du jeu de données. Cette note décrit précisément le modèle et chaque étape de calcul, en renvoyant au code source (`model/dpbb_vi.py`).

Contents

1	Introduction et énoncé du problème	1
2	Notations	2
3	Du comptage de reads au VAF attendu	2
4	Représentation sur grille et émission Beta-Binomiale	2
5	Modèle génératif : mélange de processus de Dirichlet tronqué	3
6	Inférence variationnelle en champ moyen (CAVI)	3
6.1	Famille variationnelle et ELBO	3
6.2	Mises à jour par coordonnées	3
6.3	La borne ELBO, terme à terme	4
6.4	Initialisation et redémarrages	4
7	La composante de queue neutre $1/f$	4
8	Inférence de la pureté (1A)	5
9	Sélection du nombre de clusters par fusion ICL	5
10	Post-traitement et sorties	6
11	Correspondance avec les sous-défis SMC-Het	6

12 Complexité et déterminisme	7
A Rappels utiles	7
B Visualisation : SFS postérieur-prédictif	7
C Correspondance symboles $\leftrightarrow$ code	7

1 Introduction et énoncé du problème

Un échantillon tumoral *bulk* est un mélange de cellules normales et d'une ou plusieurs populations tumorales (*sous-clones*), chacune définie par l'ensemble des mutations qu'elle porte. Chaque mutation somatique n est observée à travers b_n reads portant l'allèle variant sur une profondeur totale d_n . La *fréquence allélique variante* (VAF) observée, b_n/d_n , dépend de trois facteurs : la fraction de cellules tumorales portant la mutation — sa *cancer cell fraction* (CCF) —, le nombre de copies local (*copy number*), et la *pureté* ρ de l'échantillon (fraction de cellules tumorales). La reconstruction sous-clonale inverse cette relation : regrouper les mutations qui partagent une CCF (un clone ou sous-clone), estimer la CCF de chaque groupe, et récupérer la pureté.

Le défi DREAM SMC-Het évalue exactement ces quantités. Le tableau 1 les liste, avec la réponse fournie par DECODE_2_2.

Sous-défi	Question	Réponse de DECODE_2_2
1A	pureté de l'échantillon	ρ inféré par l'ELBO
1B	nombre de clusters	K' après sélection
1C	taille + prévalence cellulaire des clusters	comptes par cluster, $\phi_k \rho$
2A	assignation dure par mutation	$\hat{z}_n = \arg \max_k r_{n,k}$
2B	matrice de co-clustering (soft)	$\mathbf{R} \mathbf{R}^\top$

Table 1: Les sous-défis SMC-Het et les sorties correspondantes.

2 Notations

N	nombre de mutations	ρ	pureté (fraction de cellules tumorales)
b_n	reads variants de la mutation n	$\phi \in (0, 1]$	CCF (cancer cell fraction)
d_n	profondeur (reads totaux)	$\xi_n(\phi)$	VAF attendu (§3)
c_n	copy number total au locus	G	taille de la grille de CCF (= 100)
m_n	multiplicité (copies mutées)	K	troncature DP ($K_{\max} = 12$)
s	précision Beta-Binomiale (= 200)	α	concentration DP (= 1)
π_k	poids de mélange du cluster k	$r_{n,k}$	responsabilité (resp. douce)
$\Phi_{k,g}$	loi de CCF du cluster k sur la grille	N_k	masse du cluster k , $\sum_n r_{n,k}$

Table 2: Symboles principaux.

3 Du comptage de reads au VAF attendu

Pour une mutation n de copy number total c_n et de multiplicité m_n (nombre de copies chromosomiques tumorales portant la mutation), dans un échantillon de pureté ρ , une population de CCF ϕ produit un **VAF attendu**

$$\xi_n(\phi) = \frac{\rho m_n \phi}{\rho c_n + 2(1 - \rho)} \quad (1)$$

Le dénominateur est la ploïdie moyenne au locus (les c_n copies tumorales pondérées par ρ , plus 2 copies normales pondérées par $1 - \rho$) ; le numérateur compte les copies mutées. La valeur est bornée à $(10^{-6}, 1 - 10^{-6})$ pour la stabilité numérique. Inverser (1) fournit la CCF par maximum de vraisemblance d’une mutation,

$$\hat{\phi}_n = \frac{b_n}{d_n} \cdot \frac{\rho c_n + 2(1 - \rho)}{\rho m_n}, \quad (2)$$

utilisée à l’initialisation (§6.4) et pour la visualisation. Implémentation : `_expected_vaf (model/dpbb_vi.py)`.

4 Représentation sur grille et émission Beta-Binomiale

L’émission Beta-Binomiale n’est pas conjuguée à un prior sur ϕ . Deux idées la rendent calculable.

(i) **Grille de CCF (PyClone-VI).** On discrétise ϕ sur une grille fixe

$$\phi_g = \frac{g - \frac{1}{2}}{G}, \quad g = 1, \dots, G, \quad (3)$$

et la CCF de chaque cluster devient une *catégorielle* sur cette grille. La log-vraisemblance d’émission par mutation et par point de grille est précalculée une seule fois sous forme d’une matrice $N \times G$,

$$\text{LL}[n, g] = \log p(b_n \mid d_n, \xi_n(\phi_g)), \quad (4)$$

partagée par tous les clusters : un cluster ne diffère que par sa loi $\Phi_{k,\cdot}$ sur la grille. C’est la source principale de la rapidité du moteur. Implémentation : `_emission_loglik`.

(ii) **Émission Beta-Binomiale.** Avec une précision s (sur-dispersion des reads), on pose $\alpha_n = \xi_n s$ et $\beta_n = (1 - \xi_n)s$ et

$$\log p(b_n \mid d_n, \xi_n) = \log B(b_n + \xi_n s, d_n - b_n + (1 - \xi_n)s) - \log B(\xi_n s, (1 - \xi_n)s) + \text{cste}, \quad (5)$$

où $\log B(x, y) = \ln \Gamma(x) + \ln \Gamma(y) - \ln \Gamma(x + y)$ et la constante $\log \binom{d_n}{b_n}$ est omise (sans effet sur l’optimisation). Quand $s \rightarrow \infty$ on retrouve l’émission *binomiale* $b_n \log \xi_n + (d_n - b_n) \log(1 - \xi_n)$; mais une binomiale sous-estime la dispersion et tend à *sur-découper*, d’où le choix $s = 200$ par défaut.

5 Modèle génératif : mélange de processus de Dirichlet tronqué

On utilise une construction *stick-breaking* (Sethuraman, 1994) du processus de Dirichlet, tronquée à $K = K_{\max}$ composantes :

$$v_k \sim \text{Beta}(1, \alpha), \quad k = 1, \dots, K, \quad (6)$$

$$\pi_k = v_k \prod_{j < k} (1 - v_j), \quad (7)$$

$$\phi_k \sim \text{Cat}(\text{grille de CCF}), \quad (8)$$

$$z_n \sim \text{Cat}(\boldsymbol{\pi}), \quad (9)$$

$$b_n \mid z_n, \phi \sim \text{BetaBin}(d_n, \xi_n(\phi_{z_n}), s). \quad (10)$$

La troncature borne le nombre de composantes à K , mais le prior stick-breaking pousse les poids des composantes inutilisées vers ≈ 0 : le nombre *effectif* de clusters est appris, non fixé. Le prior sur ϕ_k est uniforme sur la grille, $p(\phi_k = \phi_g) = 1/G$.

6 Inférence variationnelle en champ moyen (CAVI)

6.1 Famille variationnelle et ELBO

On approxime le postérieur par la loi factorisée

$$q(\mathbf{z}, \mathbf{v}, \phi) = \prod_{n=1}^N q(z_n) \prod_{k=1}^K q(v_k) \prod_{k=1}^K q(\phi_k), \quad (11)$$

avec $q(z_n) = \text{Cat}(r_{n,\cdot})$, $q(v_k) = \text{Beta}(a_k, b_k)$ et $q(\phi_k) = \text{Cat}(\Phi_{k,\cdot})$. On maximise la borne inférieure de l'évidence (ELBO),

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(\mathbf{b}, \mathbf{z}, \mathbf{v}, \phi)] - \mathbb{E}_q[\log q], \quad (12)$$

par montée par coordonnées : chaque facteur est optimisé en forme close, les autres étant fixés.

6.2 Mises à jour par coordonnées

Posons l'*évidence attendue* de la mutation n pour le cluster Beta k ,

$$\text{Elik}_{n,k} = \mathbb{E}_{q(\phi_k)}[\log p(b_n | \phi_k)] = \sum_g \Phi_{k,g} \text{LL}[n, g] = (\text{LL} \Phi^\top)_{n,k}. \quad (13)$$

CCF des clusters $q(\phi_k)$. La mise à jour est un softmax sur la grille de la somme des log-vraisemblances pondérée par les responsabilités :

$$\log \Phi_{k,g} \propto \log \frac{1}{G} + \sum_n r_{n,k} \text{LL}[n, g], \quad \Phi_{k,\cdot} = \text{softmax}_g \left(\log \frac{1}{G} + (\mathbf{R}^\top \text{LL})_{k,g} \right). \quad (14)$$

La CCF ponctuelle d'un cluster s'en déduit par $\phi_k = \sum_g \Phi_{k,g} \phi_g$.

Poids stick-breaking $q(v_k)$. Avec $N_k = \sum_n r_{n,k}$,

$$a_k = 1 + N_k, \quad b_k = \alpha + \sum_{j>k} N_j, \quad (15)$$

et les log-poids attendus suivent des identités digamma (ψ) :

$$\mathbb{E}[\log v_k] = \psi(a_k) - \psi(a_k + b_k), \quad \mathbb{E}[\log(1 - v_k)] = \psi(b_k) - \psi(a_k + b_k), \quad \mathbb{E}[\log \pi_k] = \mathbb{E}[\log v_k] + \sum_{j<k} \mathbb{E}[\log(1 - v_j)]. \quad (16)$$

Responsabilités $q(z_n)$.

$$r_{n,k} \propto \exp \left(\mathbb{E}[\log \pi_k] + \text{Elik}_{n,k} \right), \quad \sum_k r_{n,k} = 1. \quad (17)$$

6.3 La borne ELBO, terme à terme

En injectant les facteurs dans (12) et en utilisant le prior uniforme sur ϕ_k , l'ELBO se décompose en

$$\mathcal{L} = \underbrace{\sum_{n,k} r_{n,k} (\mathbb{E}[\log \pi_k] + \text{Elik}_{n,k})}_{\text{données + assignation}} + \underbrace{\sum_n \mathcal{H}[q(z_n)]}_{\text{entropie de } z} + \underbrace{\sum_k \mathcal{H}[q(\phi_k)]}_{\text{terme CCF}} + K \log \frac{1}{G} - \underbrace{\sum_k \text{KL}(q(v_k) \parallel \text{Beta}(1, \alpha))}_{\text{KL stick-breaking}}, \quad (18)$$

Algorithm 1 CAVI pour un redémarrage (`_fit_once`)

Require: $LL \in \mathbb{R}^{N \times G}$, troncature K , concentration α

- 1: $\hat{\phi}_n \leftarrow \phi_{\arg \max_g LL[n, g]}$ {CCF-MLE par mutation, (2)}
 - 2: centres $\leftarrow$ k-means++($\hat{\phi}$, K) ; initialiser Φ par des gaussiennes autour des centres
 - 3: initialiser r depuis Φ {évite l'effondrement au premier pas}
 - 4: **repeat**
 - 5: $\Phi \leftarrow \text{softmax}_g (\log \frac{1}{G} + \mathbf{R}^\top LL)$ {CCF des clusters, (14)}
 - 6: $N_k \leftarrow \sum_n r_{n, k}$; $a_k \leftarrow 1 + N_k$; $b_k \leftarrow \alpha + \sum_{j > k} N_j$; calculer $\mathbb{E}[\log \pi_k]$ (16)
 - 7: $\text{Elik} \leftarrow LL \Phi^\top$; $r_{n, k} \leftarrow \text{softmax}_k (\mathbb{E}[\log \pi_k] + \text{Elik}_{n, k})$ {(17)}
 - 8: évaluer $\mathcal{L}$ (18)
 - 9: **until** $|\Delta \mathcal{L}| < \text{tol} (|\mathcal{L}| + 1)$
 - 10: **retour** r , $\{\phi_k = \sum_g \Phi_{k, g} \phi_g\}$, $\mathcal{L}$
-

avec $\mathcal{H}[q(z_n)] = -\sum_k r_{n, k} \log r_{n, k}$ et $\mathcal{H}[q(\phi_k)] = -\sum_g \Phi_{k, g} \log \Phi_{k, g}$. La KL de Beta(a, b) à Beta(1, α) admet la forme close

$$\text{KL} = (a - 1) \mathbb{E}[\log v] + (b - \alpha) \mathbb{E}[\log(1 - v)] - (\log B(a, b)) + (\log B(1, \alpha)). \quad (19)$$

L'itération s'arrête lorsque la variation relative de $\mathcal{L}$ passe sous $\text{tol} = 10^{-5}$ (ou après `max_iter`= 400 balayages). Le pseudocode 1 résume le tout.

6.4 Initialisation et redémarrages

Un départ plat ferait s'effondrer toutes les composantes sur le pic clonal dominant. À la place, chaque redémarrage initialise les CCF des clusters par **k-means++** sur les CCF-MLE par mutation $\hat{\phi}_n$ (2) (`_kmeanspp_centers`), de sorte que les composantes démarrent à des prévalences *distinctes*. Le fit complet est répété `n_restarts`= 10 fois (graines différentes), et l'on conserve le redémarrage de **meilleur ELBO** (`fit_dpbb_vi`). C'est l'analogue variationnel du « fix multi-restart » des échantillonneurs SMC.

7 La composante de queue neutre $1/f$

Sous évolution tumorale neutre, le spectre des mutations sous-clonales suit une loi de puissance $M(f) \propto f^{-\gamma}$ (Williams *et al.*, 2016). Un mélange de Beta pures « explique » ce continuum lisse en le **sur-découpant en clusters fantômes** : c'est le mode d'échec dominant sur les échantillons simulés. On ajoute donc une composante *fixe* (la colonne $k = 0$), de loi de CCF en loi de puissance

$$g_g^{\text{tail}} \propto \phi_g^{-\text{exponent}} \quad \text{pour } \phi_g \geq \text{tail_min_ccf}, \text{ sinon } 0, \quad (20)$$

normalisée (`neutral_tail_grid`, `exponent` = 2, `tail_min_ccf` = 0.03).

Remarque. La contribution de la queue à une mutation est sa **vraie log-vraisemblance marginale**, et *non* l'espérance de champ moyen :

$$\text{Elik}_n^{\text{tail}} = \log \sum_g g_g^{\text{tail}} p(b_n | d_n, \phi_g) = \log \text{sumexp}_g (\log g_g^{\text{tail}} + LL[n, g]). \quad (21)$$

En effet, une loi diffuse perdrait toujours la comparaison de champ moyen $\mathbb{E}[\log p]$ face à une Beta piquée ; la marginale (21) corrige ce biais.

La queue absorbe les mutations neutres diffuses pour que les Beta cessent de sur-découper. Ce *n'est pas* une lignée sous-clonale : elle est retirée en post-traitement (elle ne compte pas dans 1B/1C), et ses mutations sont réaffectées au cluster Beta le plus proche (elle contribue donc encore à 2A/2B).

8 Inférence de la pureté (1A)

La pureté est d'abord *semée* : soit depuis le fichier Battenberg (`cellularity_ploidy.txt`), soit estimée depuis les SNV diploïdes clonaux où $\text{VAF} \approx \rho/2$ (`estimate_purity`). Avec `purity_search`, on l'*infère* ensuite en ajustant la VI sur une grille de ρ autour du seed et en gardant la valeur qui **maximise l'ELBO** — une borne inférieure de $\log p(\text{données} \mid \rho)$:

$$\hat{\rho} = \arg \max_{\rho \in [\text{seed} \pm \text{span}]} \mathcal{L}(\text{fit}(\rho)), \quad (\text{purity_steps}=7 \text{ points}). \quad (22)$$

$\hat{\rho}$ est la prédiction du sous-défi 1A — un sous-produit, non une entrée (`steps/01_cluster.py`).

9 Sélection du nombre de clusters par fusion ICL

Après le fit, on choisit le nombre de clusters par **fusion agglomérative guidée par l'ICL** (Integrated Completed Likelihood ; Baudry *et al.*, 2010, `_icl_merge`). On part des composantes Beta retenues, et l'on *fusionne itérativement la paire qui maximise l'ICL*, jusqu'à ce qu'aucune fusion ne l'améliore. Pour un état à K clusters de poids π_k , de responsabilités $\mathbf{R}$ et de CCF ponctuelles ϕ_k , on définit

$$\text{ICL} = \underbrace{\sum_n \log \sum_k \pi_k p(b_n \mid \phi_k)}_{\text{log-vraisemblance observée}} - \underbrace{w_{\text{pen}} \frac{1}{2} \left[(K-1) \log N + \sum_k \log N_k \right]}_{\text{pénalité « N-aware »}} - \underbrace{w_{\text{ent}} \left(- \sum_{n,k} r_{n,k} \log r_{n,k} \right)}_{\text{entropie d'assignation}} \quad (23)$$

où $p(b_n \mid \phi_k)$ est évaluée à la CCF *ponctuelle* du cluster (le point de grille le plus proche), et $N_k = \sum_n r_{n,k}$.

Pourquoi la CCF ponctuelle. Fusionner deux CCF réellement distinctes produit une CCF intermédiaire qui n'explique bien ni l'une ni l'autre : la chute de log-vraisemblance *bloque* la fusion. À l'inverse, deux composantes sur-découpées (CCF quasi égales) fusionnent librement. C'est le remplacement principié d'un seuil de fusion fixe.

Pénalité « N-aware ». Les $K-1$ poids de mélange sont informés par tout l'échantillon (terme $\log N$), mais la CCF de chaque cluster n'est informée que par les N_k mutations qui lui sont assignées (terme $\log N_k$). Sans cela, un petit sous-clone réel ($N_k \approx 200$) serait pénalisé par $\log N$ ($N \approx 10^5$) dans une grande tumeur clonale-dominée — la cause du sous-clustering de l'ICL à grand N .

Entropie tempérée $w_{\text{ent}} = \text{icl_ent_w} = 0.25$. À plein poids ($w_{\text{ent}} = 1$), l'entropie d'assignation fusionne aussi les *vrais* sous-clones clairsemés (peu de mutations, faible profondeur $\Rightarrow$ assignation ambiguë $\Rightarrow$ forte entropie). Or, une fois la queue neutre (§7) en place pour absorber le continuum, cette protection anti-sur-découpage est *redondante*. La tempérer ($w_{\text{ent}} < 1$) découple les deux rôles : la queue rejette le continuum, une entropie allégée n'arrête que les vrais sur-découpages. Un balayage sur la cohorte montre un optimum net à 0.25, qui fait passer le score 1B de ≈ 0.79 à **0.808** sans coût sur 2A/1C (le terme $w_{\text{pen}} = \text{icl_pen_w}$ tempère analogiquement la pénalité, façon postérieur grossi de Miller–Dunson, 2019).

10 Post-traitement et sorties

`postprocess` transforme le fit brut en clustering final :

1. **Retirer la queue** (colonne 0) et mémoriser sa masse $\text{tail_frac} = \frac{1}{N} \sum_n r_{n,0}$.

Algorithm 2 Fusion agglomérative guidée par l'ICL (`_icl_merge`)

Require: responsabilités $\mathbf{R}$, CCF $\{\phi_k\}$, matrice LL

```
1:  $\text{cur} \leftarrow \text{ICL}(\mathbf{R}, \{\phi_k\})$  {(23)}
2: while le nombre de clusters  $> 1$  do
3:   for all paires  $(j, \ell)$  do
4:     CCF fusionnée  $\phi_m \leftarrow (\pi_j \phi_j + \pi_\ell \phi_\ell) / (\pi_j + \pi_\ell)$ 
5:     responsabilités fusionnées : colonne  $r_{\cdot j} + r_{\cdot \ell}$ 
6:     gain  $\leftarrow \text{ICL}(\text{état fusionné}) - \text{cur}$ 
7:   end for
8:   if le meilleur gain  $\leq 0$  then
9:     break
10:  end if
11:  appliquer la meilleure fusion ;  $\text{cur} += \text{gain}$ 
12: end while
13: retour  $\mathbf{R}, \{\phi_k\}$ 
```

2. **Élaguer** les composantes Beta de masse $N_k < \max(1, \text{mass_floor_frac} \cdot N)$ ($\text{mass_floor_frac} = 0.005$).
3. **Sélectionner** K' par fusion ICL (Algorithme 2) ; une variante `selection="merge"` fournit une fusion par seuil de CCF fixe (*baseline*).
4. **Réétiqueter** : renormaliser les responsabilités, assigner durement $\hat{z}_n = \arg \max_k r_{n,k}$, retirer les clusters vides, puis *trier par CCF décroissante* (cluster 1 = pic clonal) pour des labels stables.

La sortie par échantillon (`results/<sid>_dpbbvi.npz`) contient : `hard` ($\hat{z}_n$), `soft` ($\mathbf{R} \in \mathbb{R}^{N \times K'}$), `ccf` (ϕ_k), `counts` (N_k), `purity` ($\hat{\rho}$), `n_clusters` (K') et `elbo`.

11 Correspondance avec les sous-défis SMC-Het

L'étape `02_scoring_files.py` aligne le clustering sur l'ordre exact des mutations du VCF de scoring, puis produit (cf. tableau 1) : $\mathbf{1A} = \hat{\rho}$; $\mathbf{1B} = K'$; $\mathbf{1C}$: comptes par cluster + prévalence cellulaire $\phi_k \rho$; $\mathbf{2A}$: $\hat{z}_n$ en ordre VCF ; $\mathbf{2B}$ (optionnel) : la matrice de co-clustering $\text{CCM} = \mathbf{R} \mathbf{R}^\top$ (probabilité que deux mutations partagent un cluster). Les mutations absentes de la table sont assignées au cluster clonal par défaut.

12 Complexité et déterminisme

Le coût dominant par balayage CAVI est le produit $\text{LL} \Phi^\top$ et $\mathbf{R}^\top \text{LL}$, en $O(NKG)$. La matrice d'émission LL ($O(NG)$) est calculée une seule fois. Avec $G = 100$, $K \leq 12$, et N de l'ordre de 10^3 – 10^4 , le fit d'un échantillon (10 redémarrages $\times 7$ valeurs de pureté) prend de l'ordre de la seconde à la minute. L'algorithme est *déterministe* (graines fixées) et ne dépend que de `numpy/scipy`.

A Rappels utiles

Beta-Binomiale. Pour $b \sim \text{BetaBin}(d, \alpha, \beta)$ avec $\alpha = \xi s$, $\beta = (1 - \xi)s$ et $s = \alpha + \beta$:

$$\mathbb{E}[b] = d\xi, \quad \text{Var}[b] = d\xi(1 - \xi) \frac{d + s}{s + 1}.$$

La variance du VAF b/d vaut donc $\xi(1 - \xi)(d + s)/(d(s + 1))$, qui tend vers la variance binomiale $\xi(1 - \xi)/d$ lorsque $s \rightarrow \infty$. (Ces moments servent à tracer le SFS postérieur-prédicatif, annexe B.)

Digamma. $\psi(x) = \frac{d}{dx} \ln \Gamma(x)$; pour $v \sim \text{Beta}(a, b)$, $\mathbb{E}[\log v] = \psi(a) - \psi(a + b)$.

B Visualisation : SFS postérieur-prédicatif

Le script `steps/04_plots.py` reproduit la figure SFS de DECODE : l’histogramme du spectre observé (VAF ou CCF) recouvert du modèle ajusté, décomposé en aires colorées empilées par cluster. Pour le cluster k (CCF ponctuelle ϕ_k , poids $\pi_k = N_k/N$), le nombre attendu de mutations dans un intervalle de fréquence I est

$$\widehat{\text{count}}_k(I) = \pi_k \sum_{n=1}^N \Pr(\text{freq}_n \in I \mid d_n, \xi_n(\phi_k), s),$$

les probabilités d’intervalle étant obtenues par une approximation gaussienne de moments Beta-Binomiaux (annexe A) — équivalent lisse et rapide de l’émission exacte.

C Correspondance symboles $\leftrightarrow$ code

Concept	Symbole / équation	Fonction (<code>model/dpbb_vi.py</code> , sauf indication)
VAF attendu	$\xi_n(\phi)$, (1)	<code>_expected_vaf</code>
Émission	$\text{LL}[n, g]$, (4)	<code>_emission_loglik</code>
Init. k-means++	$\hat{\phi}_n$, (2)	<code>_kmeanspp_centers</code>
Boucle CAVI	Alg. 1, (18)	<code>_fit_once</code>
Redémarrages	meilleur ELBO	<code>fit_dpbb_vi</code>
Queue neutre	g^{tail} , (20)–(21)	<code>neutral_tail_grid</code>
Pureté	$\hat{\rho}$, (22)	<code>steps/01_cluster.py</code>
Score ICL	(23)	<code>_icl</code>
Fusion ICL	Alg. 2	<code>_icl_merge</code>
Post-traitement	§10	<code>postprocess</code>

Table 3: De la notation mathématique au code source.

References

- [1] A. Gillis, J. Roth. *PyClone-VI: scalable inference of clonal population structures*. BMC Bioinformatics, 2020.
- [2] J. Sethuraman. *A constructive definition of Dirichlet priors*. Statistica Sinica, 1994.
- [3] D. Blei, M. Jordan. *Variational inference for Dirichlet process mixtures*. Bayesian Analysis, 2006.
- [4] C. Biernacki, G. Celeux, G. Govaert. *Assessing a mixture model for clustering with the integrated completed likelihood*. IEEE TPAMI, 2000.
- [5] J.-P. Baudry *et al.* *Combining mixture components for clustering*. JCGS, 2010.
- [6] M. Williams *et al.* *Identification of neutral tumor evolution across cancer types*. Nature Genetics, 2016.

- [7] G. Caravagna *et al.* *Subclonal reconstruction of tumors using machine learning (MOBSTER)*. Nature Genetics, 2020.
- [8] J. Miller, D. Dunson. *Robust Bayesian inference via coarsening*. JASA, 2019.
- [9] A. Salcedo *et al.* *A community effort to create standards for evaluating tumor subclonal reconstruction (SMC-Het / DREAM)*. Nature Biotechnology, 2020.